

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МОЕВМ**

**ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ**  
**по дисциплине «Введение в нереляционные базы данных»**  
**Тема: Информационная система компании по производству и монтажу**  
**кухонных гарнитуров**

|                  |       |                 |
|------------------|-------|-----------------|
| Студент гр. 3344 | _____ | Тукалкин В.А    |
| Студент гр. 3344 | _____ | Святохо В.В.    |
| Студент гр. 3344 | _____ | Щербак М.С.     |
| Преподаватель    | _____ | Заславский М.М. |

Санкт-Петербург

2026

## **ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ**

Студент Тукалкин В.А.

Студент Святохо В.В.

Студент Щербак М.С.

Группа 3344

Тема работы: "Информационная система компании по производству и монтажу кухонных гарнитуров"

Исходные данные:

- Основной акцент - прототип системы управления производством (заказы, раскрой, рабочие часы, логистика).
- Предлагаемая СУБД: MongoDB
- Сложность: Сложная
- Набор данных/API: Синтетические данные

Содержание пояснительной записки:

1. Содержание
2. Введение
3. Сценарий использования
4. Модель данных
5. Разработанное приложение
6. Выводы
7. Приложения
8. Список использованных источников

Предполагаемый объем пояснительной записки:

Не менее 16 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 25.05.2026

Дата защиты реферата: 25.05.2026

|                  |       |                 |
|------------------|-------|-----------------|
| Студент гр. 3344 | _____ | Тукалкин В.А    |
| Студент гр. 3344 | _____ | Святохо В.В.    |
| Студент гр. 3344 | _____ | Щербак М.С.     |
| Преподаватель    | _____ | Заславский М.М. |

## АННОТАЦИЯ

В рамках индивидуального домашнего задания разработана информационная система компании по производству и монтажу кухонных гарнитуров «Kitchify». Приложение представляет собой веб-сайт с тремя ролями: клиент, рабочий и администратор. Клиент может выбрать дизайн, материал и цвет кухни, оформить заказ и отслеживать его статус по этапам (раскрой, производство, доставка, монтаж, завершён). Рабочий видит список задач, может взять их в работу и завершить. Администратор управляет сотрудниками, выполняет массовый импорт/экспорт данных в формате JSON, а также контролирует заказы. Для хранения данных выбрана нереляционная СУБД MongoDB, обеспечивающая гибкость схемы (вложенные этапы заказа) и высокую скорость чтения. Backend реализован на FastAPI (Python) с асинхронным драйвером Motor, аутентификация — JWT. Frontend написан на React (TypeScript) с использованием Vite, Material UI и React Query. Все компоненты контейнеризированы с помощью Docker. В репозитории представлены тестовые данные и отладочные пользователи. Приложение демонстрирует полный цикл обработки заказа — от выбора дизайна до передачи задач рабочим и контроля со стороны администратора.

Ссылка на проект: <https://github.com/moevm/nsql1h26-kitch>

## **СОДЕРЖАНИЕ**

|      |                                                     |    |
|------|-----------------------------------------------------|----|
| 1.   | ВВЕДЕНИЕ                                            | 6  |
| 1.1. | Актуальность решаемой проблемы                      | 6  |
| 1.2. | Постановка задачи                                   | 6  |
| 1.3. | Предлагаемое решение                                | 7  |
| 1.4. | Качественные требования к решению                   | 7  |
| 2.   | СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ                              | 8  |
| 2.1. | Макет UI                                            | 8  |
| 2.2. | Сценарий использования для импорта данных           | 14 |
| 2.3. | Сценарий использования для представления данных     | 16 |
| 2.4. | Сценарий использования для анализа данных           | 17 |
| 2.5. | Сценарий использования для экспорта данных          | 18 |
| 2.6. | Выводы                                              | 18 |
| 3.   | МОДЕЛЬ ДАННЫХ                                       | 19 |
| 3.1. | Нереляционная модель данных                         | 19 |
| 3.2. | Реляционная модель данных                           | 34 |
| 3.3. | Сравнение моделей                                   | 43 |
| 4.   | РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ                            | 47 |
| 4.1. | Краткое описание                                    | 47 |
| 4.2. | Использованные технологии                           | 47 |
| 4.3. | Снимки экрана приложения                            | 47 |
| 5.   | ВЫВОДЫ                                              | 49 |
| 5.1. | Достигнутые результаты                              | 49 |
| 5.2. | Недостатки и пути для улучшения полученного решения | 50 |
| 5.3. | Будущее развитие решения                            | 50 |
| 6.   | ПРИЛОЖЕНИЯ                                          | 51 |
| 6.1. | Документация по сборке и развертыванию приложения   | 51 |
| 6.2. | Инструкция для пользователя                         | 51 |
| 7.   | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ                    | 54 |

## **1. ВВЕДЕНИЕ**

### **1.1. Актуальность решаемой проблемы**

Рынок индивидуального производства и монтажа кухонной мебели постоянно растёт. Клиенты хотят не просто купить готовую кухню, а получить изделие, полностью соответствующее их планировке, стилю и бюджету. При этом заказчикам необходим удобный инструмент для выбора дизайна, материалов, цвета, а также для отслеживания статуса выполнения заказа. Со стороны компании важно эффективно управлять заказами, распределять задачи между рабочими (раскрой, производство, доставка, монтаж) и контролировать складские остатки материалов. Существующие решения часто не предоставляют единой платформы, объединяющей клиента, производственный персонал и администратора. Поэтому создание информационной системы, автоматизирующей весь жизненный цикл заказа на изготовление кухни, является актуальной задачей.

### **1.2. Постановка задачи**

Задача проекта – разработать веб-приложение для компании по производству и монтажу кухонных гарнитуров, которое позволяет:

- клиентам – регистрироваться, выбирать дизайн и материалы, оформлять заказ, отслеживать его статус по этапам (раскрой, производство, доставка, монтаж, завершён);
- рабочим – просматривать список своих задач, начинать и завершать этапы работы, видеть дедлайны;
- администраторам – управлять заказами, сотрудниками, выполнять массовый импорт/экспорт данных, а также просматривать финансовую сводку (в перспективе).

Приложение должно обеспечивать надёжное хранение данных (MongoDB), производительный API (FastAPI) и удобный пользовательский интерфейс (React). Развёртывание осуществляется через Docker.

### **1.3. Предлагаемое решение**

- Backend: FastAPI (Python) с асинхронными эндпоинтами, JWT-аутентификация, интеграция с MongoDB через Motor.
- Frontend: React (TypeScript) + Vite + Material UI, клиентские хуки для работы с API (React Query), адаптивный дизайн.
- База данных: MongoDB – выбрана из-за гибкости схемы (вложенные документы для этапов заказа) и простоты масштабирования.
- Контейнеризация: Docker Compose объединяет три сервиса – db (MongoDB), backend, frontend (nginx).

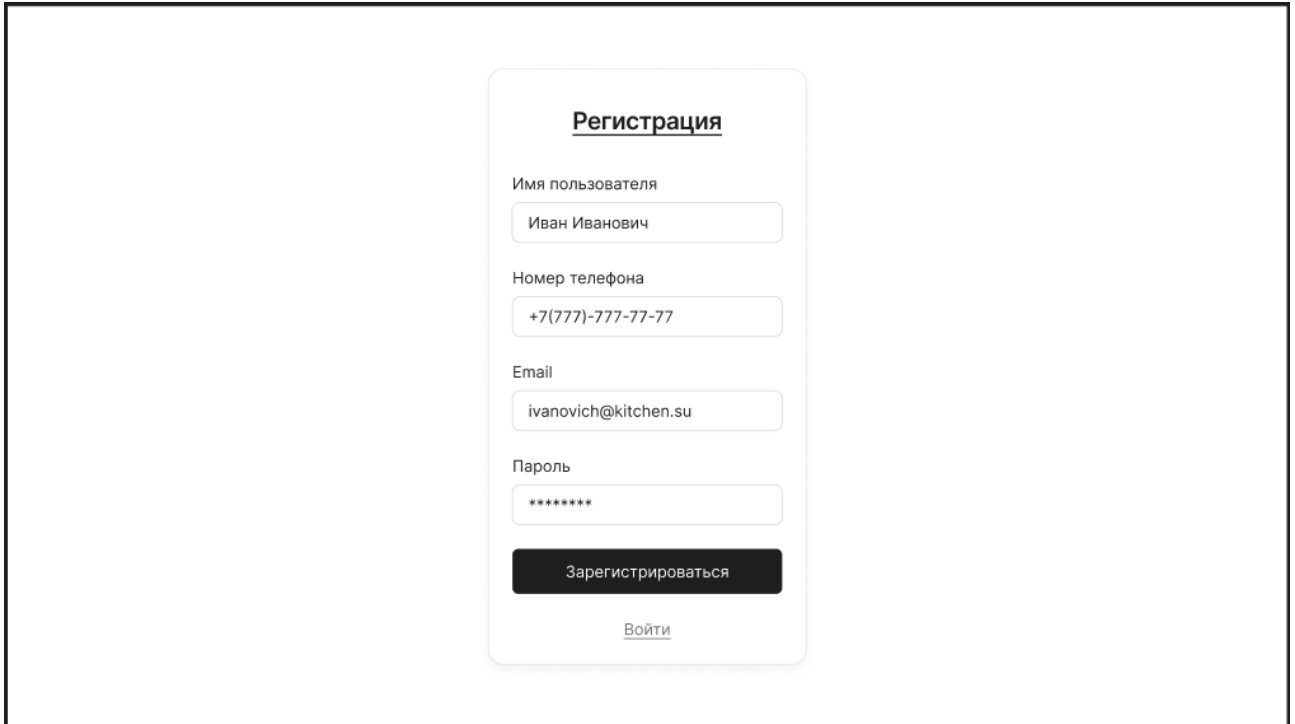
### **1.4. Качественные требования к решению**

Решение должно быть удобным, производительным, надежным и легко расширяемым, с высокой степенью взаимодействия между пользователями, удобным механизмом поиска и фильтрации, а также возможностью быстрого развертывания на различных платформах благодаря использованию Docker

## 2. СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ

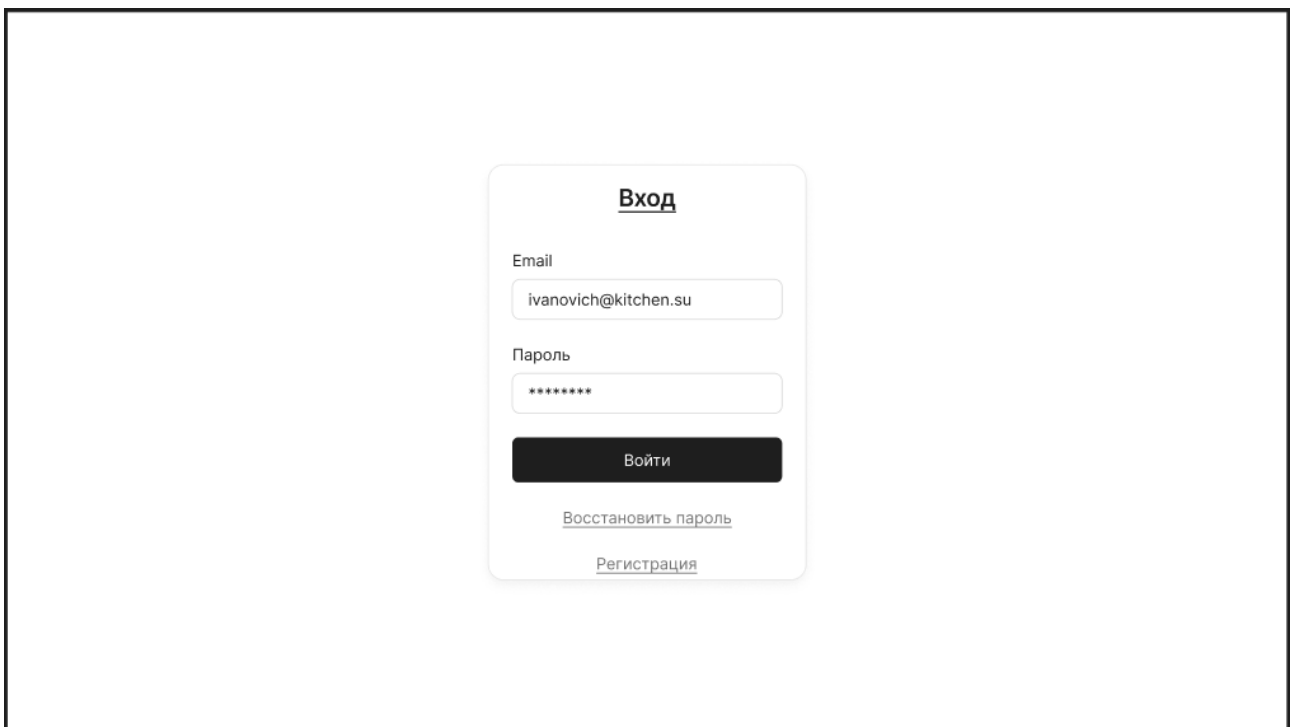
### 2.1. Макет UI

На рисунках 1-14 представлен макет приложения



The image shows a mobile application registration screen. At the top, the title "Регистрация" is centered and underlined. Below it are four input fields: "Имя пользователя" (containing "Иван Иванович"), "Номер телефона" (containing "+7(777)-777-77-77"), "Email" (containing "ivanovich@kitchen.su"), and "Пароль" (containing "\*\*\*\*\*"). A dark button labeled "Зарегистрироваться" is positioned below the password field. At the bottom, there is a link labeled "Войти".

Рисунок 1 – Макет страницы регистрации



The image shows a mobile application login screen. At the top, the title "Вход" is centered and underlined. Below it are two input fields: "Email" (containing "ivanovich@kitchen.su") and "Пароль" (containing "\*\*\*\*\*"). A dark button labeled "Войти" is positioned below the password field. Below the button are two links: "Восстановить пароль" and "Регистрация".

Рисунок 2 – Макет страницы авторизации



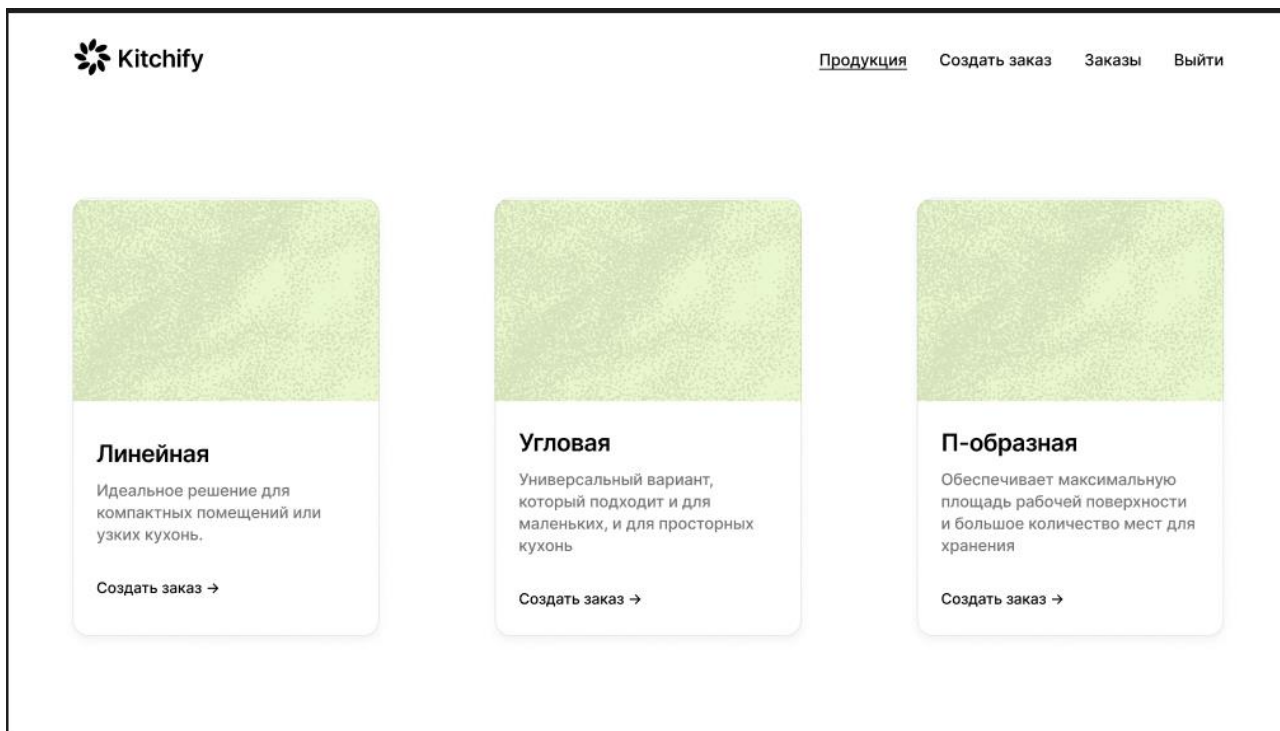


Рисунок 3 – Макет страницы с продукцией для Покупателя

The screenshot shows the Kitchify website header with the logo and navigation links: [Продукция](#), [Создать заказ](#), [Заказы](#), and [Выйти](#). Below the header, there is a large form titled "Информация по заказу" with a subtitle "Укажите нам свои предпочтения". The form contains several input fields and dropdown menus for user information and order details.

**Информация по заказу**  
Укажите нам свои предпочтения

Имя заказчика:     Номер телефона:     Адрес доставки:

Тип кухни:     Цвет:     Материал:     Этаж:     Лифт:

Дополнительные пожелания:

Линейная №10: 100000Р    Массив дерева: 50000Р    Доставка 5 этаж: 500Р

[Оплатить: 150500Р](#)

Рисунок 4 – Макет страницы создания заказа для Покупателя

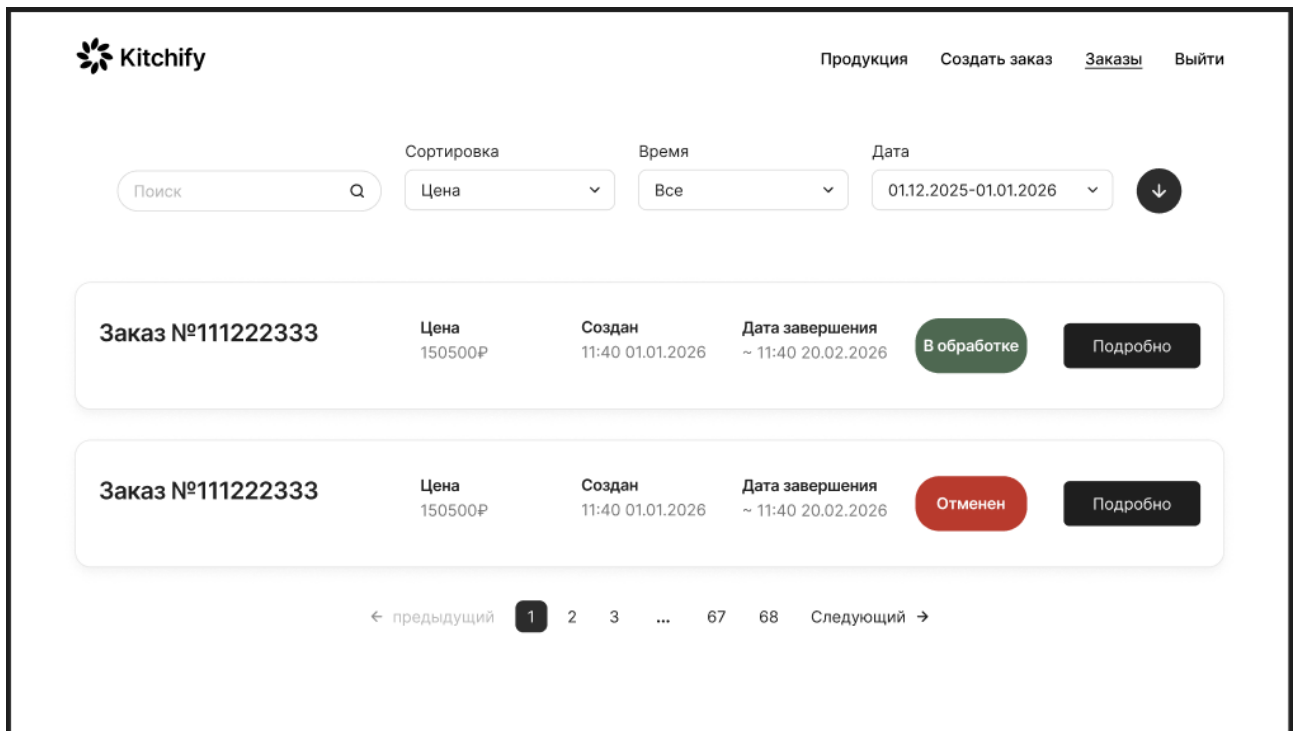


Рисунок 5 – Макет страницы отслеживания заказов для Покупателя

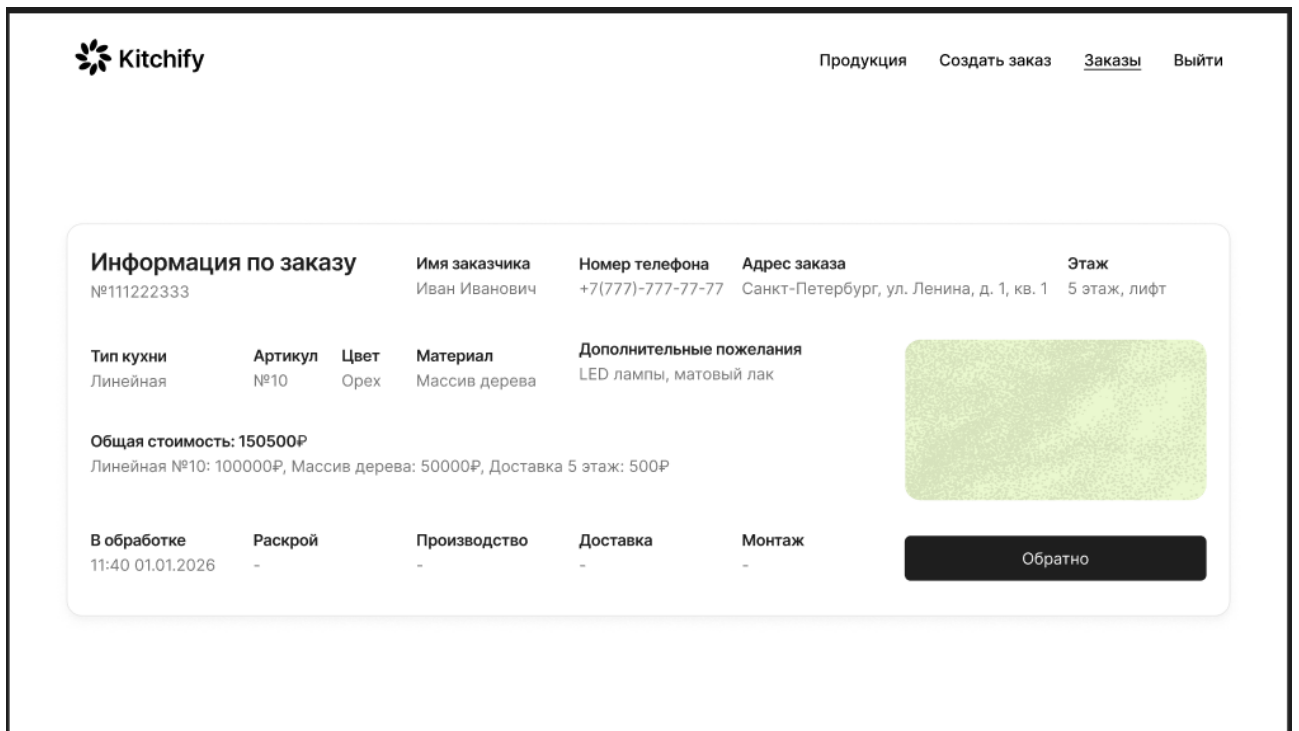


Рисунок 6 – Макет страницы подробной информации о заказе для Покупателя

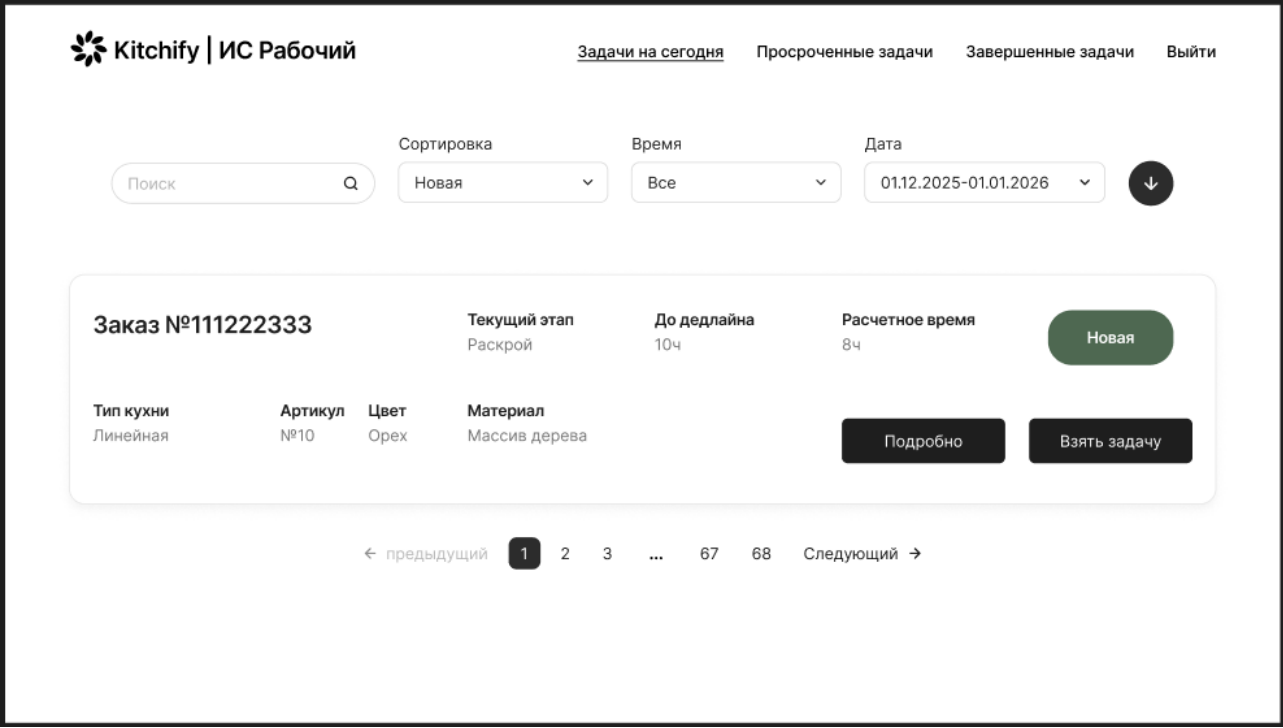


Рисунок 7 – Макет страницы задач для Рабочего

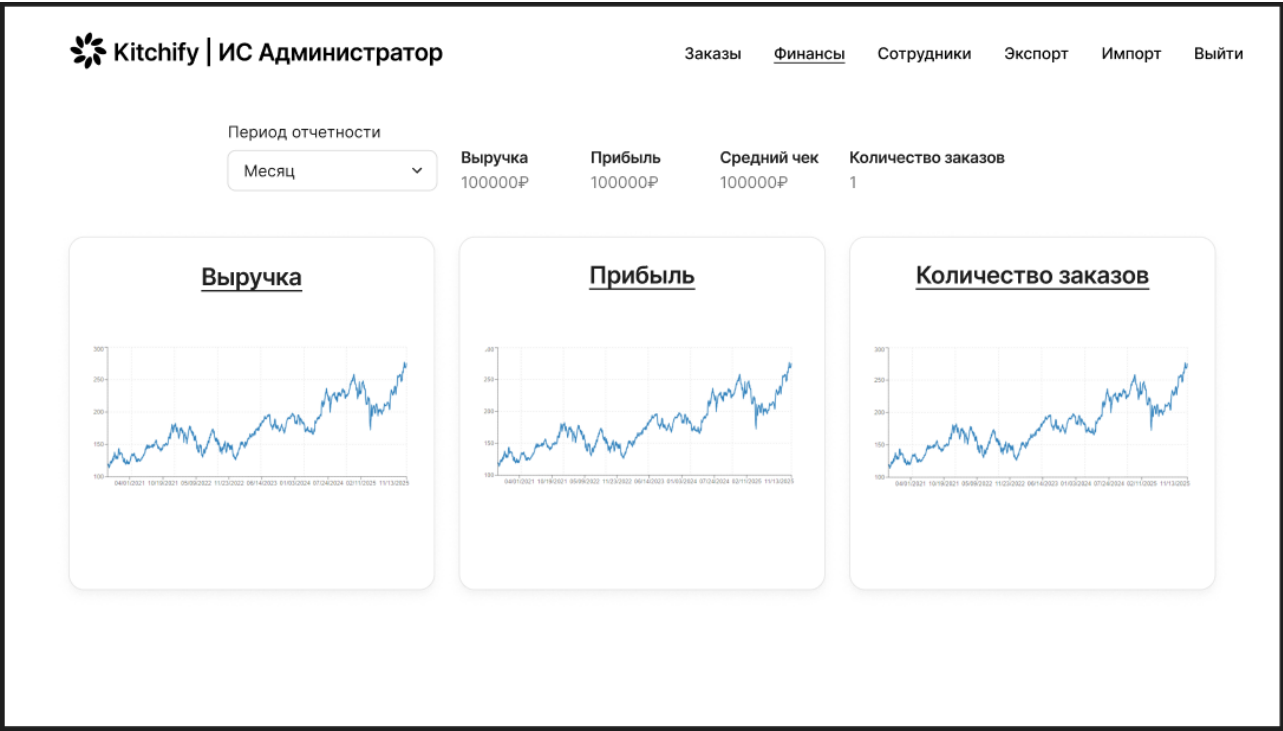


Рисунок 8 – Макет страницы графиков с финансами для Админа

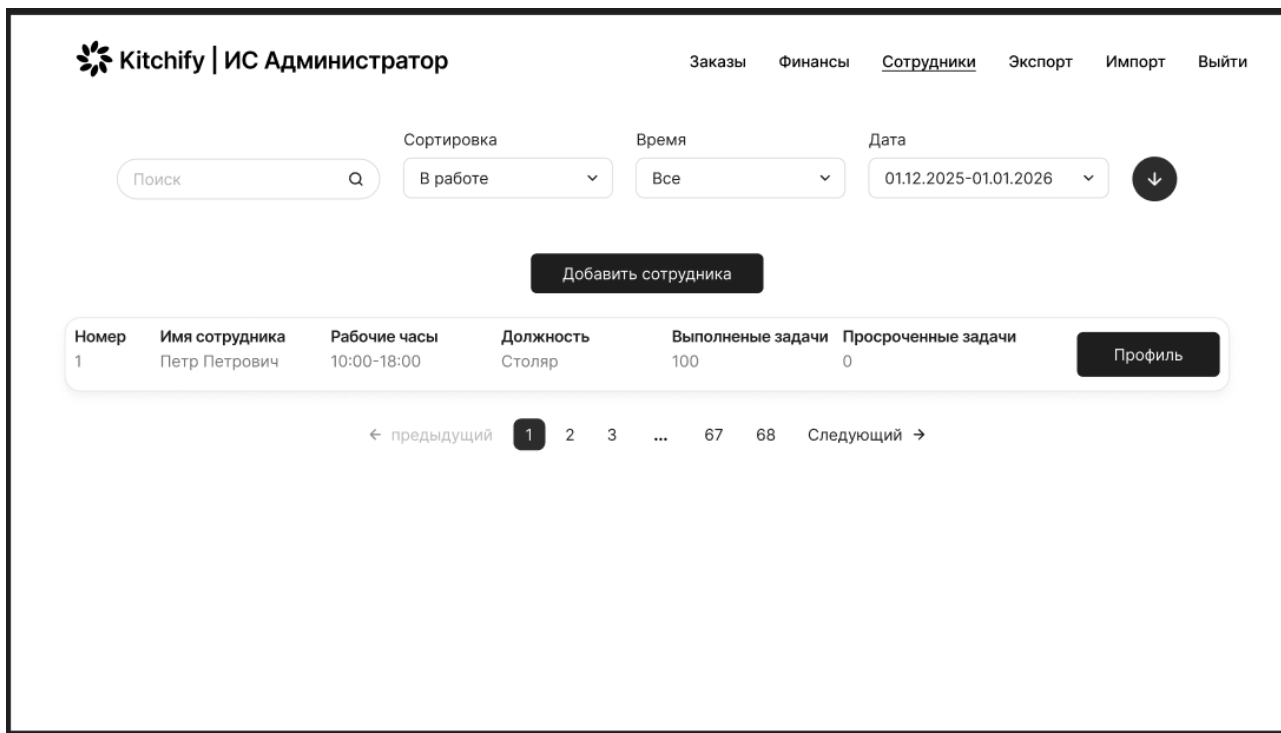


Рисунок 9 – Макет страницы со списком сотрудников для Админа

The modal form is titled 'Добавить сотрудника' with a close button 'x' in the top right corner. It contains the following fields: 'Имя сотрудника' (filled with 'Петр Петрович'), 'Email' (filled with 'petrovich@kitchen.su'), 'Должность' (filled with 'Столяр'), 'Начало смены' (filled with '10:00'), and 'Конец смены' (filled with '18:00'). At the bottom of the form is a 'Добавить' button.

Рисунок 10 – Макет модального окна добавления сотрудника для Админа

Заказы

Финансы

Сотрудники

Экспорт

Импорт

Выйти

Информация о сотруднике

Имя сотрудника

№1

Петр Петрович

Возраст

56

Стаж

25

Дата создания

01.01.2026

Дата изменения

01.02.2026

Email сотрудника

petrpetrovich@kitchen.su

Дата рождения

01.01.1970

Начало смены

10:00

Конец смены

18:00

Должность

Столяр

Комментарий

Хорошо работает

История должностей

Столяр (21.01.2025), Стажер (21.01.2024)

Сохранить

Уволить

Рисунок 11 – Макет страницы с подробной информацией о сотруднике для Админа

| ИС Администратор

Заказы

Финансы

Сотрудники

Экспорт

Импорт

Выйти

Поиск

Сортировка

В работе

Время

Все

Дата

01.12.2025-01.01.2026

Добавить сотрудника

| Номер | Имя сотрудника | Рабочие часы | Должность | Выполненные задачи | Просроченные задачи | Профиль |
|-------|----------------|--------------|-----------|--------------------|---------------------|---------|
| 1     | Петр Петрович  | 10:00-18:00  | Столяр    | 100                | 0                   |         |

| Задача     | Этап         | Расчетное время | Затраченное время | Просрочил на |
|------------|--------------|-----------------|-------------------|--------------|
| №111222333 | Производство | 8ч              | 6ч                |              |
| №111222334 | Производство | 8ч              | 10ч               | 2ч           |

Готово

Просрочена

← Предыдущий

1

2

3

...

67

68

Следующий →

← Предыдущий

1

2

3

...

67

68

Следующий →

Рисунок 12 – Макет страницы для просмотра задач сотрудника для Админа



Кitchify | ИС Администратор

[Заказы](#)
[Финансы](#)
[Сотрудники](#)
[Экспорт](#)
[Импорт](#)
[Выйти](#)

Период отчетности

Месяц

Статус заказа

В работе

Заказ №111222333

Дата создания

01.01.2026

Цена

250000₽

Рабочий на заказе

Петр Петрович

Имя заказчика

Иван Иванович

Номер телефона

+7(777)-777-77-77

Email

ivanovich@kitchen.su

Адрес заказа

Санкт-Петербург, ул. Ленина, д. 1, кв. 1

Статус заказа

Производство

Текущий этап

Производство

Статус задачи

В работе

Раскрой

Производство

Доставка

Монтаж

Сохранить

Отменить заказ

Рисунок 13 – Макет страницы подробностей о заказе для Админа



Кitchify | ИС Администратор

[Заказы](#)
[Финансы](#)
[Сотрудники](#)
[Экспорт](#)
[Импорт](#)
[Выйти](#)

Период отчетности

Месяц

Статус заказа

Отменен

Заказ №111222333

Дата создания

01.01.2026

Цена

250000₽

Отменен

Имя заказчика

Иван Иванович

Номер телефона

+7(777)-777-77-77

Email

ivanovich@kitchen.su

Адрес заказа

Санкт-Петербург, ул. Ленина, д. 1, кв. 1

Рисунок 14 – Макет страницы заказов для Админа

## 2.2. Сценарий использования для импорта данных

### 2.2.1. Массовый импорт данных

#### Сценарий "UC-7.1: Импорт"

14

Действующие лица: Администратор, Система

Цели:

Администратор: импортировать данные.

Основной сценарий:

- 1) Администратор открывает главный dashboard.
- 2) Администратор нажимает кнопку "Импорт".
- 3) Система вызывает диалоговое окно, в котором администратор выбирает файл JSON для импорта.

Результат: Администратор импортирует данные в систему.

Альтернативный сценарий: При выборе неподдерживаемого формата для файла система выведет сообщение об ошибке при загрузке.

### **2.2.2. Импорт данных вручную**

Сценарий "UC-5.2: Добавить нового рабочего"

Действующие лица: Администратор, Система

Цели:

Администратор: добавить нового рабочего

Основной сценарий:

- 1) Администратор открывает раздел «Сотрудники».
- 2) Администратор может настраивать фильтры для удобного поиска, искать по названиям и переключать страницы сотрудников.
- 3) Администратор нажимает «Добавить сотрудника».
- 4) Администратор вводит:
  - ФИО рабочего
  - Должность
  - email сотрудника
  - время работы
  - Возраст
  - Стаж

5) Система присваивает уникальный номер рабочему, создаёт логин и пароль.

Результат: Администратор добавляет нового сотрудника.

## **2.3. Сценарий использования для представления данных**

### **2.3.1. Поиск и таблицы**

Сценарий "UC-2.2: Просмотр статуса заказа"

Действующие лица: Покупатель, Система

Цели:

Покупатель: отследить текущее состояние своего заказа.

Основной сценарий:

- 1) Покупатель авторизуется в системе и переходит в раздел «Мои заказы».
- 2) Система отображает список заказов с указанием номера, даты создания и текущего статуса.
- 3) Покупатель может настраивать фильтры для удобного поиска, искать по названиям и переключать страницы заказов.
- 4) Покупатель выбирает конкретный заказ.
- 5) Система показывает детальную информацию:
  - Номер заказа
  - Имя заказчика
  - Общую стоимость
  - Дата создания
  - Тип кухни
  - Артикул
  - Дополнительные пожелания
  - Материал
  - Адрес доставки
  - Этаж для доставки



- Статус (В обработке, Раскрой, Производство, Доставка, Монтаж, Завершен, Отменен)

- История изменений статуса с датами

- Запланированная дата завершения

Результат: Покупатель информирован о ходе выполнения заказа.

### **2.3.2. Графики**

Сценарий "UC-6.1: Просмотр финансовой сводки и статистики"

Действующие лица: Администратор, Система

Цели:

Администратор: увидеть ключевые финансовые показатели бизнеса.

Основной сценарий:

1) Администратор открывает главный dashboard (или раздел «Финансы»).

2) Система отображает:

- Выручку за период (день/неделя/месяц/сезон/год/выбранный период)

- Прибыль (выручка минус затраты)

- Количество выполненных заказов

- Средний чек

- График динамики выручки

3) Администратор может переключать периоды для анализа.

Результат: Администратор видит текущее финансовое состояние бизнеса.

### **2.4. Сценарий использования для анализа данных**

Сценарий "UC-6.1: Просмотр финансовой сводки и статистики"

Действующие лица: Администратор, Система

Цели:

Администратор: увидеть ключевые финансовые показатели бизнеса.

Основной сценарий:

1) Администратор открывает главный dashboard (или раздел «Финансы»).

2) Система отображает:

- Выручку за период (день/неделя/месяц/сезон/год/выбранный период)
- Прибыль (выручка минус затраты)
- Количество выполненных заказов
- Средний чек
- График динамики выручки

3) Администратор может переключать периоды для анализа.

Результат: Администратор видит текущее финансовое состояние бизнеса.

## **2.5. Сценарий использования для экспорта данных**

Сценарий "UC-7.2: Экспорт"

Действующие лица: Администратор, Система

Цели:

Администратор: экспортировать данные.

Основной сценарий:

- 1) Администратор открывает главный dashboard.
- 2) Администратор нажимает кнопку "Экспорт".
- 3) Система вызывает диалоговое окно, в котором администратор сохраняет файл JSON.

Результат: Администратор экспортирует данные из системы.

## **2.6. Выводы**

В системе преобладают операции чтения (клиенты просматривают каталог и свои заказы, рабочие – список задач). Операции записи (создание заказа, взятие/завершение задачи, добавление сотрудника) выполняются реже. Такой баланс оправдывает использование MongoDB с денормализацией для ускорения чтения.

### 3. МОДЕЛЬ ДАННЫХ

#### 3.1. Нереляционная модель данных

На рисунке 15 представлена графическое представление для модели MongoDB.

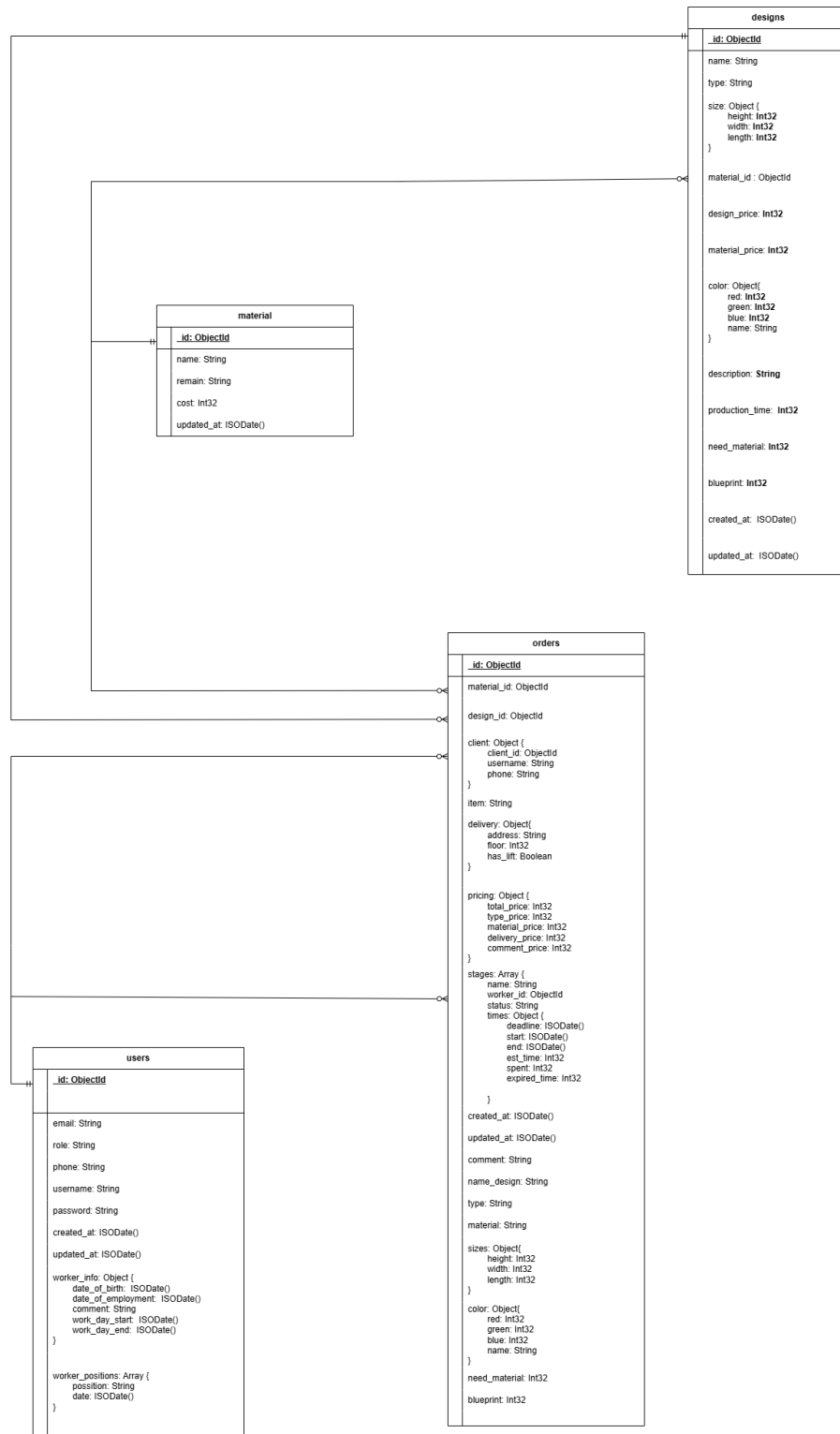


Рисунок 15 – графическое представление модели для MongoDB

## Описание назначений коллекций, типов данных и сущностей

### users

Назначение: хранит информацию о пользователях системы (клиенты, работники, администраторы). Для работников внутри документа хранится профильная информация и история должностей, что позволяет избежать JOIN-ов.

Типы данных полей:

- `_id` - уникальный идентификатор пользователя (ObjectId, 12 байт)
- `email` - электронная почта (~100 байт, переменный)
- `role` - роль пользователя (строка: 'client', 'worker', 'admin') (~10 байт)
- `phone` - номер телефона (~15 байт, может отсутствовать)
- `username` - имя пользователя (~50 байт, переменный)
- `password` - хеш пароля (~60 байт, переменный)
- `created_at` - дата и время создания (ISODate, 8 байт)
- `updated_at` - дата и время последнего обновления (ISODate, 8 байт)
- `worker_info` - вложенный объект с профилем работника

(присутствует только если `role = 'worker'`):

- `date_of_birth` - дата рождения (ISODate, 8 байт)
- `date_of_employment` - дата приёма на работу (ISODate, 8 байт)
- `comment` - комментарий (~100 байт, может отсутствовать)
- `work_day_start` - время начала смены (ISODate, 8 байт)
- `work_day_end` - время окончания смены (ISODate, 8 байт)

- `worker_positions` - массив исторических должностей (присутствует

только для работников):

- `position` - название должности (~50 байт)
- `date` - дата назначения (ISODate, 8 байт)

## designs

Назначение: коллекция для хранения информации о материалах, используемых в дизайн-проектах. Объединяет в себе данные о материале и связанном с ним дизайне (в отличие от реляционной модели, где эти сущности разделены).

Типы данных полей:

- `_id` - уникальный идентификатор дизайна (ObjectId, 12 байт)
- `name` - название дизайна (~50 байт, переменный)
- `type` - тип кухни (например, 'ЛДСП', 'МДФ') (~20 байт)
- `size` - вложенный объект с размерами:
  - `height` - высота (Int32, 4 байта)
  - `width` - ширина (Int32, 4 байта)
  - `length` - длина (Int32, 4 байта)
- `material_id` - идентификатор ссылки на материал (ObjectId, 12 байт, может отсутствовать)
- `material` - название материала (~50 байт, переменный)
- `design_price` - стоимость дизайна (Int32, 4 байта)
- `material_price` - стоимость материала в этом дизайне (Int32, 4 байта)
- `color` - вложенный объект с цветом:
  - `red` - красная компонента (Int32, 4 байта)
  - `green` - зелёная компонента (Int32, 4 байта)
  - `blue` - синяя компонента (Int32, 4 байта)
  - `name` - название цвета (~30 байт)
- `description` - описание (~200 байт, может отсутствовать)
- `production_time` - время производства (Int32, 4 байта)
- `need_material` - необходимое количество материала (Int32, 4 байта)
- `blueprint` - ссылка на чертёж (Int32, 4 байта, может отсутствовать)
- `created_at` - дата и время создания (ISODate, 8 байт)
- `updated_at` - дата и время последнего обновления (ISODate, 8 байт)

## **material**

Назначение: справочник материалов с остатками на складе и себестоимостью. Упрощённая версия по сравнению с предыдущей, содержит только основные характеристики.

Типы данных полей:

- `_id` - уникальный идентификатор материала (ObjectId, 12 байт)
- `name` - название материала (~50 байт, переменный)
- `remain` - остаток на складе (4 байт)
- `cost` - себестоимость единицы (Int32, 4 байта)
- `updated_at` - дата и время последнего обновления (ISODate, 8 байт)

## **orders**

Назначение: хранит полную информацию о заказе, включая данные клиента, доставку, цены, этапы выполнения, характеристики дизайна и материалы.

Типы данных полей:

- `_id` - уникальный идентификатор заказа (ObjectId, 12 байт)
- `client` (вложенный объект):
  - `client_id` - идентификатор клиента из коллекции `users` (int, 8 байт)
  - `username` - имя клиента (~50 байт, дублируется для быстрого доступа)
  - `phone` - телефон клиента (~15 байт, дублируется)
- `item` - наименование товара/услуги (~100 байт, переменный)
- `comment` - комментарий к заказу (строка, ~200 байт, может отсутствовать)
- `delivery` (вложенный объект):
  - `floor` - этаж доставки (int, 4 байта)
  - `has_lift` - наличие лифта (boolean, 1 байт)

- address - адрес доставки (~100 байт)
- price (вложенный объект):
  - total\_price - итоговая стоимость заказа (int, 4 байта)
  - type\_price - стоимость типа кухни (int, 4 байта)
  - material\_price - стоимость материалов (int, 4 байта)
  - delivery\_price - стоимость доставки (int, 4 байта)
  - comment\_price - стоимость комментария/особых пожеланий (int, 4 байта, фиксированная сумма при наличии комментария)
- stages (массив):
  - name - название этапа (например, "Раскрой", "Производство", "Доставка") (~50 байт)
  - status - статус этапа (enum: "pending", "in\_progress", "completed", "cancelled") (~15 байт)
  - worker\_id - идентификатор ответственного работника (ObjectId, 12 байт, может отсутствовать)
  - deadline - крайний срок выполнения (ISODate, 8 байт)
  - estimated\_time - оценочное время выполнения в минутах (int, 4 байта)
  - spent\_time - фактически затраченное время в минутах (int, 4 байта, может отсутствовать)
  - expired\_time - время просрочки в минутах (int, 4 байта, может отсутствовать)
  - start - фактическое время начала (ISODate, 8 байт, может отсутствовать)
  - end - фактическое время окончания (ISODate, 8 байт, может отсутствовать)
- name\_design - название выбранного дизайн-проекта (дублируется из коллекции designs) (~50 байт)
- type - тип кухни (например, "Линейная", "П-образная") (~30 байт, берётся из designs, если не заполнено пользователем)

- sizes (вложенный объект, берётся из designs, если не заполнено пользователем):
  - height - высота (int, 4 байта)
  - width - ширина (int, 4 байта)
  - length - длина (int, 4 байта)
- material\_id - идентификатор материала (ObjectId, 12 байт, берётся из designs, если не заполнено пользователем)
- material - название материала (дублируется из коллекции materials) (~50 байт, берётся из designs, если не заполнено пользователем)
- color (вложенный объект, берётся из designs, если не заполнено пользователем):
  - r - красная компонента (int, 4 байта)
  - g - зелёная компонента (int, 4 байта)
  - b - синяя компонента (int, 4 байта)
  - name - название цвета (~30 байт)
- need\_material - необходимое количество материала на изготовление (int, 4 байта, берётся из designs, если не заполнено пользователем)
- blueprint - номер чертежа в физической библиотеке (int, 4 байта, берётся из designs, если не заполнено пользователем)
- created\_at - дата и время создания заказа (ISODate, 8 байт)
- updated\_at - дата и время последнего изменения заказа (ISODate, 8 байт)

### **Оценка объёма информации, хранимой в модели**

Для вычислений введены следующие параметры (аналогично реляционной модели):

$C = 1000$  — количество клиентов

$W = 20$  — количество работников

$A = 3$  — количество администраторов



O = 500 – количество заказов

D = 50 – количество дизайн-проектов

M = 30 – количество материалов

K\_stages = 5 – количество этапов на один заказ

K\_positions = 1.2 – среднее количество должностей на одного работника

Размеры одной записи (в байтах) получены из оценок полей.

Коллекция users

Для клиентов и администраторов документ не содержит worker\_info и worker\_positions.

Размер одного документа:

- \_id: 12
- email: 100
- role: 10
- phone: 15
- username: 50
- password: 60
- created\_at: 8
- updated\_at: 8

Сумма: 263 байта.

Для работника добавляются поля:

worker\_info – вложенный объект:

- date\_of\_birth: 8
- date\_of\_employment: 8
- comment: 100
- work\_day\_start: 8
- work\_day\_end: 8

Итого полей: 132 байта. Служебные расходы на объект:  $\sim 4$  байта. = 136 байт.

worker\_positions – массив (среднее количество элементов K\_positions = 1.2):

Один элемент:

- position: 50
- date: 8

Сумма:  $58 + \text{служебные на элемент} (\sim 8) = 66$  байт.

Для 1.2 элементов:  $1.2 * 66 \approx 79$  байт.

Служебные расходы на массив:  $\sim 8$  байт. = 87 байт.

Итого для работника:  $263 + 136 + 87 = 486$  байт.

Количество работников:  $W = 20$ , остальных пользователей:  $C + A = 1000 + 3 = 1003$ .

Средний размер документа:

$(1003 * 263 + 20 * 486) / 1023 \approx 267$  байт

Размер коллекции:

$1023 * 267 \approx 273\,141$  байт

Коллекция designs

Поля:

- \_id: 12
- name: 50
- type: 20
- size: { height: 4, width: 4, length: 4 } + служебные (4)  $\rightarrow 16$
- material\_id: 12
- material: 50
- design\_price: 4
- material\_price: 4

- color: { r: 4, g: 4, b: 4, name: 30 } + служебные (4) → 46
- description: 200
- production\_time: 4
- need\_material: 4
- blueprint: 4
- created\_at: 8
- updated\_at: 8

Сумма полей: 442 байта. Добавим служебные накладные расходы на сам документ (~8) = 450 байт.

Размер коллекции при D = 50:

$$50 * 450 = 22\,500 \text{ байт}$$

Коллекция material

Поля:

- \_id: 12
- name: 50
- remain: 4
- cost: 4
- updated\_at: 8

Сумма: 78 байт.

Размер коллекции при M = 30:

$$30 * 78 = 2\,340 \text{ байт}$$

Коллекция orders

Один документ включает вложенные объекты client, delivery, price, sizes, color и массив stages из K\_stages = 5 элементов.

Расчёт размера одного документа приведён в табл. 1:

*Таблица 1 – расчёт размера одного документа коллекции orders*

| Поле / структура | Составляющие | Размер |
|------------------|--------------|--------|
|------------------|--------------|--------|

|               |                                                                                                                                                                                                                                                                                     |        |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|               |                                                                                                                                                                                                                                                                                     | (байт) |
| _id           | ObjectId                                                                                                                                                                                                                                                                            | 12     |
| client        | client_id (8) + username (50) + phone (15) +<br>служебные (4)                                                                                                                                                                                                                       | 77     |
| item          | строка                                                                                                                                                                                                                                                                              | 100    |
| comment       | строка                                                                                                                                                                                                                                                                              | 200    |
| delivery      | floor (4) + has_lift (1) + address (100) +<br>служебные (4)                                                                                                                                                                                                                         | 109    |
| price         | 5 полей по 4 байта + служебные (4)                                                                                                                                                                                                                                                  | 24     |
| stages        | массив из 5 элементов:– на элемент: name<br>(50) + status (15) + worker_id (12) + deadline<br>(8) + estimated_time (4) + spent_time (4) +<br>expired_time (4) + start (8) + end (8) +<br>служебные (~8) = 121 байт– массив: 5*121<br>+ служебные (8) = 613 613name_design<br>строка | 50     |
| type          | строка                                                                                                                                                                                                                                                                              | 30     |
| sizes         | height (4) + width (4) + length (4) +<br>служебные (4)                                                                                                                                                                                                                              | 16     |
| material_id   | ObjectId                                                                                                                                                                                                                                                                            | 12     |
| material      | строка                                                                                                                                                                                                                                                                              | 50     |
| color         | r (4) + g (4) + b (4) + name (30) + служебные<br>(4)                                                                                                                                                                                                                                | 46     |
| need_material | Int32                                                                                                                                                                                                                                                                               | 4      |
| blueprint     | Int32                                                                                                                                                                                                                                                                               | 4      |
| created_at    | ISODate                                                                                                                                                                                                                                                                             | 8      |
| updated_at    | ISODate                                                                                                                                                                                                                                                                             | 8      |
| Итого         |                                                                                                                                                                                                                                                                                     | 1363   |

Размер коллекции при  $O = 500$ :

$$500 * 1363 = 681\,500 \text{ байт}$$

Суммарный размер базы данных

$$273\,141 + 22\,500 + 2\,340 + 681\,500 = 979\,481 \text{ байт}$$

$$979\,481 \text{ байт} \approx 956,5 \text{ КБ} \approx 0,93 \text{ МБ}$$

Итоговая формула

$$\text{modelSize} \approx 1363 \cdot O + 450 \cdot D + 78 \cdot M + 263 \cdot (C+W+A) + (136 + 66 \cdot K_{\text{positions}}) \cdot W$$

Итоговая формула от одной переменной

$$\text{modelSize}(O) = 1363 \cdot O + 279\,729$$

Примеры запросов

Регистрация нового покупателя

```
db.users.insertOne({
  role: "client",
  username: "newuser",
  email: "newuser@example.com",
  phone: "+7 999 888-77-66",
  password: "hashed_password",
  created_at: new Date(),
  updated_at: new Date()
})
```

Оформление заказа

```
db.orders.insertOne({
  client: {
    client_id: 1,
    username: "ivan_ivanov",
    phone: "+7 999 123-45-67"
  },
  comment: "Без комментариев",
  delivery: {
```

```

    address: "г. Москва, ул. Пушкина, д. 1"
  },
  color: { main: "белый" },
  material_id: ObjectId("67e1a2b3c4d5e6f7a8b9c0d4"),
  material: "ЛДСП 16 мм",
  name_design: "Кухня \"Комфорт\"",
  type: "Прямая",
  created_at: new Date(),
  updated_at: new Date()
})

```

### Начало выполнения задачи

```

db.orders.updateOne(
  { _id: ObjectId("67e1a2b3c4d5e6f7a8b9c0d5"), "stages.name": "Раскрой",
    "stages.start": null },
  { $set: { "stages.$.start": new Date(), "stages.$.status":
    "in_progress" } }
)

```

### Добавить нового рабочего

```

let workerId = db.users.insertOne({
  role: "worker",
  username: "new_worker",
  email: "new@company.ru",
  phone: null,
  password: "hashed",
  created_at: new Date(),
  updated_at: new Date(),
  worker_info: {
    date_of_birth: new Date("1995-02-20"),
    date_of_employment: new Date("2025-03-01"),
    comment: null,
    work_day_start: new Date("1970-01-01T09:00:00Z"),
    work_day_end: new Date("1970-01-01T18:00:00Z")
  },
  worker_positions: [
    { position: "Монтажник", date: new Date("2025-03-01") }
  ]
}).insertedId

```

### Финансовая сводка за период

```

db.orders.aggregate([
  {
    $match: {
      created_at: {
        $gte: ISODate("2025-03-01T00:00:00Z"),
        $lt: ISODate("2025-04-01T00:00:00Z")
      }
    }
  },
  {
    $group: {
      _id: null,
      total_revenue: { $sum: "$price.total_price" },
      orders_count: { $sum: 1 }
    }
  }
])

```

**Вывести 10 самых эффективных за месяц работников**

```

db.orders.aggregate([
  { $unwind: "$stages" },

  { $match: {
    "stages.status": "completed",
    "stages.end": {
      $gte: ISODate("2025-03-01T00:00:00Z"),
      $lt: ISODate("2025-04-01T00:00:00Z")
    }
  }},

  {
    $group: {
      _id: "$stages.worker_id",
      completed_stages: { $sum: 1 }
    }
  },

  {
    $sort: { completed_stages: -1 }
  },

  { $limit: 10 },

```

```

{
  $lookup: {
    from: "users",
    localField: "_id",
    foreignField: "_id",
    as: "worker"
  }
},
{ $unwind: "$worker" },
{
  $project: {
    worker_id: "$_id",
    username: "$worker.username",
    completed_stages: 1,
    _id: 0
  }
}
})

```

### Вывести все заказы с просроченным этапом

```

db.orders.aggregate([
  { $unwind: "$stages" },
  {
    $match: {
      "stages.status": { $ne: "Выполнен" },
      "stages.deadline": { $lt: new Date() }
    }
  },
  {
    $group: {
      _id: "$_id",
      client: { $first: "$client" },
      delivery: { $first: "$delivery" },
      created_at: { $first: "$created_at" },
      overdue_stages: { $push: "$stages" }
    }
  },
  {
    $project: {

```



```
    order_id: "$_id",
    client_name: "$client.username",
    delivery_address: "$delivery.address",
    created_at: 1,
    overdue_stages: 1,
    _id: 0
  }
}
1)
```

### 3.2. Реляционная модель данных

На рисунке 16 представлена графическое представление для модели PostgreSQL.

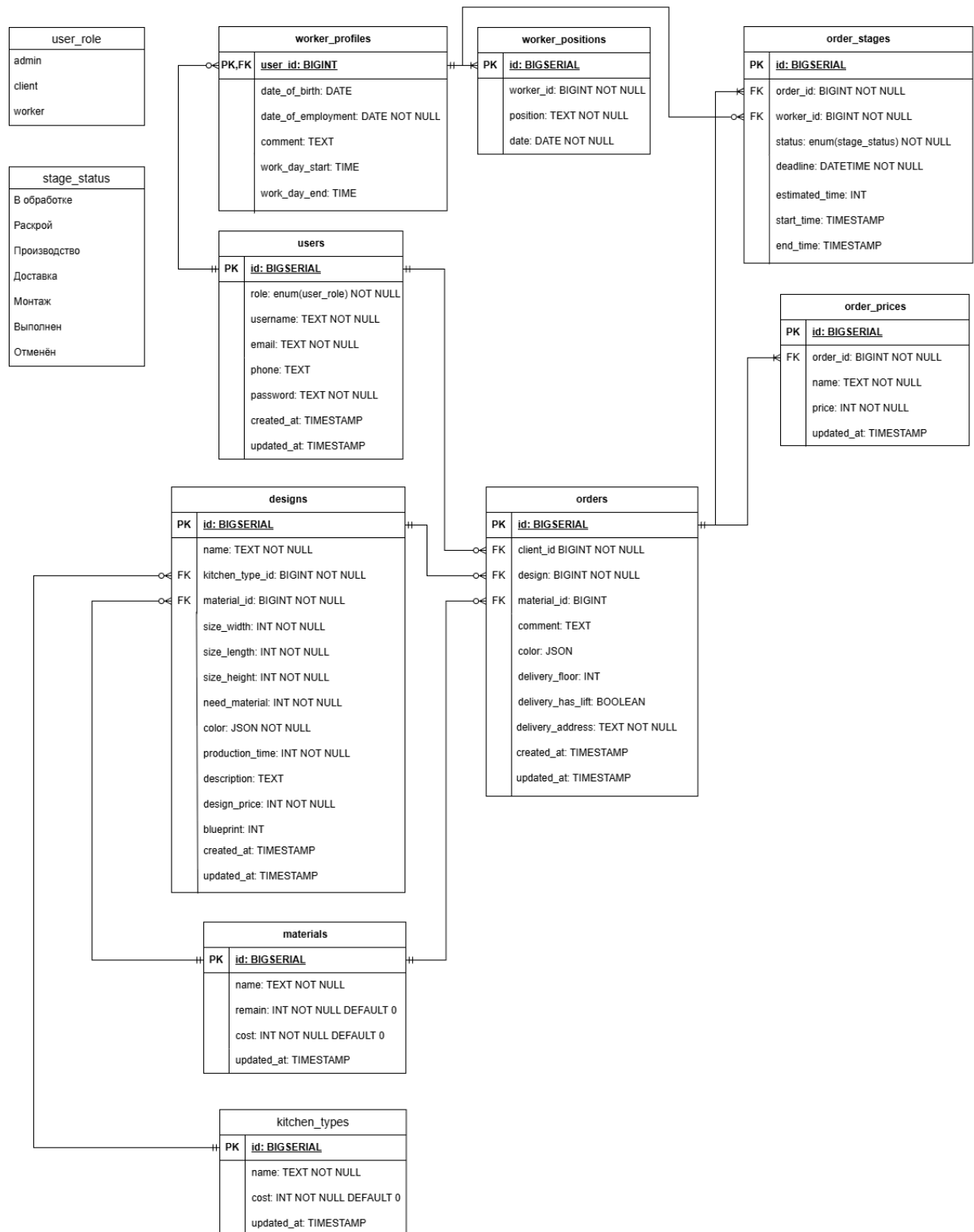


Рисунок 16 – графическое представление модели для PostgreSQL

## **Описание назначений коллекций, типов данных и сущностей**

### **user\_role**

Назначение: enum для хранения роли пользователя.

### **stage\_status**

Назначение: enum для хранения этапа заказа.

### **users**

Назначение: хранит информацию о пользователе.

Типы данных полей:

- id - уникальный идентификатор пользователя (8 байт)
- role - значение enum из user\_role (4 байт)
- username - имя пользователя (~30 байт, переменный)
- email - почта пользователя (~30 байт, переменный)
- phone - номер телефона (15 байт, может быть NULL)
- password - пароль пользователя (~36 байт, переменный)
- created\_at - время создания пользователя (8 байт)
- updated\_at - время последнего изменения пользователя (8 байт)

### **worker\_profiles**

Назначение: хранит информацию о работнике.

Типы данных полей:

- user\_id - уникальный идентификатор пользователя из таблицы users (8 байт)
- date\_of\_birth - дата рождения для расчёта возраста работника (4 байт)
- date\_of\_employment - дата трудоустройства работника для расчёта стажа работы (4 байта)
- comment - комментарий о работнике (~100 байт, может быть NULL)

- work\_day\_start - время начала смены (4 байта)
- work\_day\_end - время окончания смены (4 байта)

### **worker\_positions**

Назначение: хранит историю должностей работника.

Типы данных полей:

- id - уникальный идентификатор пользователя из таблицы users (8 байт)
- worker\_id - уникальный идентификатор работника (8 байт)
- position - название должность (~20 байт, переменный)
- date - дата изменения должности (4 байта)

### **order\_stages**

Назначение: отслеживает этапы выполнения заказа, ответственных работников, сроки и фактическое время выполнения.

Типы данных полей:

- id - уникальный идентификатор этапа (8 байт)
- order\_id - идентификатор заказа (8 байт)
- worker\_id - идентификатор ответственного работника (8 байт)
- status - статус этапа (значение enum stage\_status) (4 байта)
- deadline - крайний срок выполнения (8 байт)
- estimated\_time - оценочное время выполнения в минутах (4 байта, может быть NULL)
- start\_time - фактическое время начала (8 байт, может быть NULL)
- end\_time - фактическое время окончания (8 байт, может быть NULL)

## **order\_prices**

Назначение: хранит расшифровку стоимости заказа по отдельным позициям (например, материалы, работы, дополнительные услуги).

Типы данных полей:

- id - уникальный идентификатор позиции (8 байт)
- order\_id - идентификатор заказа (8 байт)
- name - название позиции (~50 байт, переменный)
- price - стоимость позиции (4 байта)
- updated\_at - дата и время последнего изменения цены (8 байт)

## **designs**

Назначение: содержит информацию о готовых дизайн-проектах кухонь: размеры, материалы, цвета, время производства, цена и чертёж.

Типы данных полей:

- id - уникальный идентификатор дизайна (8 байт)
- name - название дизайна (~50 байт, переменный)
- kitchen\_type\_id - тип кухни (8 байт)
- material\_id - идентификатор материала (8 байт)
- size\_width - ширина (4 байта)
- size\_length - длина (4 байта)
- size\_height - высота (4 байта)
- need\_material - необходимое количество материала (4 байта)
- color - цветовое решение в формате JSON (~150 байт, переменный)
- production\_time - время производства в днях или часах (4 байта)
- description - описание дизайна (~200 байт, может быть NULL, переменный)
- design\_price - стоимость дизайна (4 байта)
- blueprint - ссылка на чертёж или идентификатор файла (4 байта, может быть NULL)

- created\_at - дата и время создания (8 байт)
- updated\_at - дата и время последнего обновления (8 байт)

## **orders**

Назначение: основная таблица заказов, связывающая клиента, выбранный дизайн, материал, цвет, параметры доставки.

Типы данных полей:

- id - уникальный идентификатор заказа (8 байт)
- client\_id - идентификатор клиента (пользователя) (8 байт)
- design - идентификатор выбранного дизайна (8 байт)
- material\_id - идентификатор материала (8 байт, может быть NULL)
- comment - комментарий к заказу (~100 байт, может быть NULL, переменный)
- color - выбранный цвет в формате JSON (~100 байт, может быть NULL, переменный)
- delivery\_floor - этаж доставки (4 байта, может быть NULL)
- delivery\_has\_lift - наличие лифта (1 байт, может быть NULL)
- delivery\_address - адрес доставки (~100 байт, переменный)
- created\_at - дата и время создания заказа (8 байт)
- updated\_at - дата и время последнего изменения заказа (8 байт)

## **materials**

Назначение: справочник материалов с остатками на складе и себестоимостью.

Типы данных полей:

- id - уникальный идентификатор материала (8 байт)
- name - название материала (~50 байт, переменный)
- remain - остаток на складе (4 байта)
- cost - себестоимость единицы (4 байта)

- updated\_at - дата и время последнего обновления (8 байт)

### **kitchen\_types**

Назначение: справочник типов кухонь (например, угловая, прямая, островная) с базовой стоимостью.

Типы данных полей:

- id - уникальный идентификатор типа (8 байт)
- name - название типа (~50 байт, переменный)
- cost - базовая стоимость типа (4 байта)
- updated\_at - дата и время последнего обновления (8 байт)

### **Оценка объёма информации, хранимой в модели**

Для вычислений введены следующие параметры:

$C = 1000$  – количество клиентов

$W = 20$  – количество работников

$A = 3$  – количество администраторов

$O = 500$  – количество заказов

$D = 50$  – количество дизайн-проектов

$M = 30$  – количество материалов

$KT = 5$  – количество типов кухонь

$I = 50$  – количество дополнительных элементов (items)

$K\_stages = 5$  – количество этапов на один заказ

$K\_prices = 4$  – среднее количество позиций в разбивке стоимости заказа

$K\_positions = 1.2$  – среднее количество должностей на одного работника

Размеры одной записи (в байтах) получены из оценок полей.

Размер таблицы users:

Размер одной записи:  $8 + 4 + 50 + 100 + 15 + 60 + 8 + 8 = 253$  байта

Количество записей (пользователей):  $C + W + A = 1000 + 20 + 3 = 1023$

Размер таблицы:  $1023 * 253 = 258\,819$  байт

Размер таблицы worker\_profiles:

Размер одной записи:  $8 + 4 + 4 + 100 + 8 + 8 = 132$  байта

Количество записей (работников):  $W = 20$

Размер таблицы:  $20 * 132 = 2\,640$  байт

Размер таблицы workers\_positions:

Размер одной записи:  $8 + 8 + 20 + 4 = 40$  байт

Количество записей :  $W * K\_positions = 20 * 1,2 = 24$

Размер таблицы:  $24 * 40 = 960$  байт

Размер таблицы orders:

Размер одной записи:  $8 + 8 + 8 + 8 + 100 + 100 + 4 + 1 + 100 + 8 + 8 = 353$   
байта

Количество записей (заказов):  $O = 500$

Размер таблицы:  $500 * 353 = 176\,500$  байт

Размер таблицы order\_stages:

Размер одной записи:  $8 + 8 + 8 + 4 + 8 + 4 + 8 + 8 = 56$  байт

Количество записей:  $O * K\_stages = 500 * 5 = 2\,500$

Размер таблицы:  $2\,500 * 56 = 140\,000$  байт

Размер таблицы order\_prices:

Размер одной записи:  $8 + 8 + 50 + 4 + 8 = 78$  байт

Количество записей:  $O * K\_prices = 500 * 4 = 2\,000$

Размер таблицы:  $2\,000 * 78 = 156\,000$  байт

Размер таблицы designs:

Размер одной записи:  $8 + 50 + 8 + 8 + 4 + 4 + 4 + 4 + 150 + 4 + 200 + 4 + 4 + 8 + 8 = 468$  байт



Количество записей (дизайн-проектов):  $D = 50$

Размер таблицы:  $50 * 468 = 23\,400$  байт

Размер таблицы materials:

Размер одной записи:  $8 + 50 + 4 + 4 + 8 = 74$  байта

Количество записей (материалов):  $M = 30$

Размер таблицы:  $30 * 74 = 2\,220$  байт

Размер таблицы kitchen\_types:

Размер одной записи:  $8 + 50 + 4 + 8 = 70$  байт

Количество записей (кол-во кухонь):  $KT = 5$

Размер таблицы:  $5 * 70 = 350$  байт

Суммарный размер базы данных

$258\,819 + 2\,640 + 960 + 176\,500 + 140\,000 + 156\,000 + 23\,400 + 2\,220 + 350$   
 $= 760\,889$  байт

$760\,889$  байт  $\approx 743,0$  КБ  $\approx 0,73$  МБ

Итоговая формула

$modelSize(O, W) = 253 \cdot (C+W+A) + 132 \cdot W + 40 \cdot W \cdot K\_positions + 353 \cdot O +$   
 $56 \cdot O \cdot K\_stages + 78 \cdot O \cdot K\_prices + 468 \cdot D + 74 \cdot M + 70 \cdot KT$

Итоговая формула от одной переменной

$modelSize(O) = 945 \cdot O + 279\,729$

## Примеры запросов

Регистрация нового покупателя

```
INSERT INTO users (role, username, email, phone, password,
created_at, updated_at)
VALUES ('client', 'newuser', 'newuser@example.com', '+7 999 888-77-
66', 'hashed_password', NOW(), NOW());
```

## Оформление заказа

```
INSERT INTO orders (client_id, design, material_id, comment, color,
delivery_address, created_at, updated_at)
VALUES (1, 1, 1, 'Без комментариев', '{"main": "белый"}'::json, 'г.
Москва, ул. Пушкина, д. 1', NOW(), NOW())
RETURNING id;
```

## Начало выполнения задачи

```
UPDATE order_stages
SET start_time = NOW()
WHERE id = 1001
AND start_time IS NULL;
```

## Добавить нового рабочего

```
INSERT INTO users (role, username, email, phone, password,
created_at, updated_at)
VALUES ('worker', 'new_worker', 'new@company.ru', NULL, 'hashed',
NOW(), NOW())
RETURNING id;

INSERT INTO worker_profiles (user_id, date_of_birth,
date_of_employment, work_day_start, work_day_end)
VALUES (5, '1995-02-20', '2025-03-01', '09:00:00', '18:00:00');

INSERT INTO workers_positions (worker_id, position, date)
VALUES (5, 'Плотник', '2025-03-01');
```

## Вывести 10 самых эффективных за месяц работников

```
SELECT
    u.id AS worker_id,
    u.username,
    COUNT(*) AS completed_stages
FROM order_stages os
JOIN users u ON os.worker_id = u.id
WHERE os.status = 'completed'
    AND os.end_time >= '2025-03-01'
    AND os.end_time < '2025-04-01'
GROUP BY u.id, u.username
ORDER BY completed_stages DESC
LIMIT 10;
```

### Вывести все заказы с просроченным этапом

```
SELECT
    o.id AS order_id,
    os.id AS stage_id,
    os.name AS stage_name,
    os.status,
    os.deadline,
    u.username AS client_name
FROM orders o
JOIN order_stages os ON o.id = os.order_id
JOIN users u ON o.client_id = u.id
WHERE os.status != 'completed'
    AND os.deadline < NOW()
ORDER BY o.id, os.deadline;
```

### 3.3. Сравнение моделей

#### Удельный объём информации

Реляционная модель имеет меньший удельный объём информации, чем нереляционная. Это связано с тем, что в MongoDB для повышения скорости чтения применяется денормализация: в документах orders дублируются данные клиента, дизайна и материала. В PostgreSQL эти данные нормализованы и хранятся без дублирования.

#### Сравнение для ключевых сущностей (при заданных параметрах):

Таблица 2 – сравнение моделей

| Сущность              | Объём в реляционной модели (SQL)                | Объём в нереляционной модели (NoSQL) |
|-----------------------|-------------------------------------------------|--------------------------------------|
| Один заказ            | $353 + 5 \times 56 + 4 \times 78 = 945$<br>байт | 1363 байта                           |
| Пользователь-клиент   | 253 байта                                       | 263 байта                            |
| Пользователь-работник | $253 + 132 + 1,2 \times 40 = 433$               | 486 байт                             |

|          |          |          |
|----------|----------|----------|
|          | байта    |          |
| Дизайн   | 468 байт | 450 байт |
| Материал | 74 байта | 78 байт  |

Общий объём всей базы данных

- SQL: 760 889 байт ( $\approx 0,73$  МБ)
- NoSQL: 979 481 байт ( $\approx 0,93$  МБ)

Реляционная модель экономит 28,7% места.

В большинстве случаев количество запросов совпадает, но для операций, затрагивающих несколько сущностей, MongoDB позволяет выполнить всё за один запрос благодаря вложенным документам. Например, добавление работника в SQL требует трёх отдельных вставок, а в MongoDB — одной.

### Количество задействованных коллекций/таблиц

Таблица 3 – количество задействованных таблиц

| Запрос                               | Задействовано таблиц (SQL)                    | Задействовано коллекций (NoSQL) |
|--------------------------------------|-----------------------------------------------|---------------------------------|
| Регистрация нового покупателя        | 1 (users)                                     | 1 (users)                       |
| Оформление заказа                    | 3 (orders, order_stages, order_prices)        | 1 (orders)                      |
| Просмотр статуса заказа              | 2 (orders, order_stages)                      | 1 (orders)                      |
| Начало выполнения задачи             | 1 (order_stages)                              | 1 (orders)                      |
| Добавление нового рабочего           | 3 (users, worker_profiles, workers_positions) | 1 (users)                       |
| Финансовая сводка                    | 2 (orders, order_prices)                      | 1 (orders)                      |
| Самые загруженные работники за месяц | 2 (order_stages, users)                       | 2 (orders, users)               |

|                     |                                 |            |
|---------------------|---------------------------------|------------|
| Проспеченные заказы | 3 (orders, order_stages, users) | 1 (orders) |
|---------------------|---------------------------------|------------|

MongoDB позволяет выполнять многие операции, работая всего с одной коллекцией, что упрощает логику приложения и снижает количество соединений с базой данных.

### **Вывод по модели данных**

На основе приведённой оценки можно заключить следующее для данного проекта:

- По удельному объёму реляционная модель компактнее: она занимает на 28,7% меньше места. Однако в современных условиях стоимость хранения данных относительно невысока, а при заданных объёмах (менее 1 МБ) эта разница не является критическим фактором.
- По количеству запросов к базе данных MongoDB демонстрирует преимущество для операций, связанных с несколькими сущностями: оформление заказа выполняется за 2 запроса против 3 в SQL, добавление рабочего — за 1 против 3. Это снижает нагрузку на сеть и упрощает код приложения.
- По числу используемых коллекций/таблиц нереляционная модель требует меньше обращений к разным структурам данных: многие юзкейсы работают с одной коллекцией, тогда как в SQL требуется объединение 2–3 таблиц. Это ускоряет разработку и уменьшает вероятность ошибок при написании запросов.

Гибкость схемы данных в MongoDB позволяет легко добавлять новые поля в документы заказа или профиль работника без миграций, что важно для проекта, который может развиваться и требовать быстрого внесения изменений.

Несмотря на то, что реляционная модель экономит место, для данного проекта ключевыми являются удобство разработки, снижение количества

запросов и простота масштабирования бизнес-логики. MongoDB позволяет хранить заказ вместе с этапами и ценами в одном документе, что естественным образом отражает предметную область и минимизирует необходимость в JOIN-ах. Кроме того, использование встроенных массивов и объектов упрощает поддержку целостности на уровне приложения и ускоряет вывод данных на клиент.

Таким образом, нереляционная модель (MongoDB) является предпочтительным выбором для данного проекта, обеспечивая более высокую скорость разработки, меньшее количество обращений к базе данных и лучшую адаптируемость к изменяющимся требованиям.

## **4. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ**

### **4.1. Краткое описание**

Приложение «Kitchify» состоит из трёх Docker-контейнеров:

1) backend – FastAPI сервер на Python. Реализует бизнес-логику: регистрация/аутентификация, управление заказами, задачи, рабочие, импорт/экспорт. Подключается к MongoDB через Motor.

2) frontend – одностраничное приложение на React (TypeScript) с использованием Vite и Material UI. Взаимодействует с backend через REST API. Роутинг настроен для трёх ролей.

3) db – MongoDB 8.0, данные хранятся в Docker-volume.

При старте backend автоматически заполняет базу тестовыми данными (скрипт seed.py): 5 клиентов, 5 рабочих, 1 админ, 6 материалов, 6 дизайнов, ~60 заказов в разных статусах.

### **4.2. Используемые технологии**

- Backend: Python 3.14, FastAPI, Motor (асинхронный драйвер MongoDB), PyJWT, passlib[argon2], python-dotenv, uvicorn.
- Frontend: TypeScript, React 19, React Router 7, TanStack Query, Axios, Material UI (MUI), SCSS, Vite.
- База данных: MongoDB 8.0.
- Контейнеризация: Docker, Docker Compose.

### **4.3. Снимки экрана приложения**

На рисунке 17 представлена схема переходов экранов приложения





## 5. ВЫВОДЫ

### 5.1. Достигнутые результаты

В ходе работы разработана информационная система компании по производству и монтажу кухонных гарнитуров «Kitchify», представляющая собой веб-приложение для автоматизации полного цикла обработки заказа – от выбора дизайна клиентом до выполнения задач рабочими и административного контроля.

Приложение ориентировано на три роли пользователей:

1) Клиент может зарегистрироваться, выбрать дизайн кухни, материал, цвет, указать параметры доставки, оформить заказ с автоматическим расчётом стоимости, а также отслеживать его статус по этапам (раскрой, производство, доставка, монтаж, завершён). Для удобства реализована фильтрация заказов по множеству критериев с сохранением параметров в URL.

2) Рабочий получает список доступных и назначенных ему задач, может взять задачу в работу и завершить её. Система автоматически фиксирует время начала и окончания этапа, при завершении создаёт следующий этап, отслеживает просрочки. Реализована фильтрация задач по статусу, этапу, дизайну, материалу и датам.

3) Администратор управляет сотрудниками: просматривает список с фильтрацией по имени, должности, рабочему времени, количеству выполненных и просроченных задач; добавляет новых работников (с генерацией пароля), редактирует их профиль и увольняет. Также администратор может выполнять массовый импорт и экспорт всех данных системы в формате JSON (коллекции users, designs, materials, orders).

Приложение полностью контейнеризировано (Docker Compose), backend написан на FastAPI (Python) с асинхронным драйвером Motor, аутентификация основана на JWT, пароли хешируются argon2. Frontend реализован на React (TypeScript) с использованием Vite, Material UI и React Query. В базе данных MongoDB при старте автоматически создаются тестовые данные (5 клиентов, 5

рабочих, 1 администратор, 6 материалов, 6 дизайнов, около 60 заказов в различных статусах), что позволяет сразу продемонстрировать все реализованные сценарии.

## **5.2. Недостатки и пути для улучшения полученного решения**

На данный момент приложение «Kitchify» имеет следующие недостатки, устранение которых позволит улучшить пользовательский опыт и функциональность:

- Ограниченные возможности редактирования заказа клиентом. Клиент не может изменить адрес доставки или состав заказа после создания.
- Улучшение интерфейса и UX. Текущий дизайн функционален, но некоторые элементы (например, отображение просроченных этапов) можно сделать более наглядными. Желательно добавить подсказки, улучшить адаптивность для мобильных устройств и реализовать тёмную тему.

## **5.3. Будущее развитие решения**

- Добавление графического отображения этапов заказа (диаграмма Ганта) для клиента.
- Интеграция с платёжными шлюзами (онлайн-оплата заказа).
- Уведомления (email / Telegram) о смене статуса заказа.
- Мобильное приложение на React Native или Flutter.

## 6. ПРИЛОЖЕНИЯ

### 6.1. Документация по сборке и разворачиванию приложения

Требования: Docker, Docker Compose, Git.

Шаги:

1) Клонировать репозиторий:

```
git clone https://github.com/moevm/nsql1h26-kitch.git
```

2) Перейти в корневую папку:

```
cd nsql1h26-kitch
```

3) Скопировать `.env.example` в `.env` (при необходимости отредактировать).

4) Собрать и запустить контейнеры:

```
docker-compose up --build -d
```

5) Дождаться запуска всех трёх сервисов (проверить `docker-compose logs`).

6) Открыть в браузере:

- Frontend: <http://127.0.0.1:8080>
- Backend API документация: <http://127.0.0.1:8000/docs>

Остановка:

```
docker-compose down (добавить -v для удаления томов с данными).
```

### 6.2. Инструкция для пользователя

Отладочные пользователи

| Роль    | Логин              | Пароль    |
|---------|--------------------|-----------|
| Клиент  | client@example.com | client123 |
| Рабочий | worker@example.com | worker123 |
| Админ   | admin@example.com  | admin123  |

#### Клиент

- Вход – заполнить email и пароль.

- Выбор дизайна – на странице /products представлены типы кухонь. Нажатие на карточку → переход к созданию заказа.
- Создание заказа – заполнить все поля (имя, телефон, адрес, дизайн, цвет, материал, этаж, лифт). Нажать «Оплатить» – заказ сохраняется, появляется в списке заказов.
- Просмотр заказов – /orders. Фильтры по дизайну, материалу, адресу, этапу, цене, датам. Карточки содержат номер, цену, дату создания, дату завершения, статус. Кнопка «Подробнее» показывает полную информацию и историю этапов.
- Отмена заказа – на странице деталей (пока не реализовано, можно через API). Планируется добавить кнопку.

### **Рабочий**

- Вход – под своей учётной записью.
- Список задач – /worker/tasks. Видны все доступные и свои задачи. Фильтры по дизайну, материалу, статусу, этапу, дате создания.
- Взять задачу – на карточке задачи с статусом «Доступна» нажать «Взять». После этого задача переходит в статус «В процессе» и привязывается к рабочему.
- Завершить задачу – когда этап выполнен, нажать «Завершить». Система автоматически создаёт следующий этап (или завершает заказ).
- Просмотр заказа – кнопка «Заказ» на карточке открывает детали заказа (только чтение).

### **Администратор**

- Вход – под учётной записью admin.
- Управление сотрудниками – /admin/employees. Список с фильтрами по имени, должности, рабочему времени, количеству выполненных/просроченных задач, дате создания. Кнопка «Добавить

сотрудника» открывает модальное окно: ФИО, email, дата рождения, должность, стаж, начало/конец смены. После создания генерируется пароль – его нужно передать работнику. Кнопка «Профиль» позволяет редактировать данные и увольнять сотрудника.

- Импорт/экспорт – кнопки в шапке. Импорт: выбрать JSON-файл (экспортированный ранее). Экспорт: скачать файл export.json со всеми коллекциями.

- Прочие разделы – «Заказы» и «Финансы» в текущей версии представляют собой заглушки, будут доработаны.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Репозиторий проекта «Kitchify» : информационная система компании по производству и монтажу кухонных гарнитуров – URL: <https://github.com/moevm/nsql1h26-kitch> (дата обращения: 24.05.2026). – Текст : электронный.
2. FastAPI : документация / авт.-сост. S. Ramirez. – URL: <https://fastapi.tiangolo.com> (дата обращения: 24.05.2026). – Текст : электронный.
3. MongoDB : официальная документация / MongoDB, Inc. – URL: <https://www.mongodb.com/docs> (дата обращения: 24.05.2026). – Текст : электронный.
4. Motor : асинхронный драйвер MongoDB для Python / A. Rosen, M. White. – URL: <https://motor.readthedocs.io> (дата обращения: 24.05.2026). – Текст : электронный.
5. React : документация / Meta Platforms, Inc. – URL: <https://react.dev> (дата обращения: 24.05.2026). – Текст : электронный.
6. Vite : инструмент сборки фронтенда / E. You. – URL: <https://vitejs.dev> (дата обращения: 24.05.2026). – Текст : электронный.
7. Material UI : библиотека компонентов для React / MUI Team. – URL: <https://mui.com/material-ui> (дата обращения: 24.05.2026). – Текст : электронный.
8. TanStack Query : управление состоянием серверных данных / T. Linsley. – URL: <https://tanstack.com/query/latest> (дата обращения: 24.05.2026). – Текст : электронный.
9. Docker : документация / Docker, Inc. – URL: <https://docs.docker.com> (дата обращения: 24.05.2026). – Текст : электронный.
10. Docker Compose : оркестрация многоконтейнерных приложений / Docker, Inc. – URL: <https://docs.docker.com/compose> (дата обращения: 24.05.2026). – Текст : электронный.

11. Python : официальная документация / Python Software Foundation. – URL: <https://docs.python.org/3> (дата обращения: 24.05.2026). – Текст : электронный.
12. PyJWT : библиотека для работы с JWT в Python / J. Padilla. – URL: <https://pyjwt.readthedocs.io> (дата обращения: 24.05.2026). – Текст : электронный.
13. Passlib : система хеширования паролей / Е. Т. Muller. – URL: <https://passlib.readthedocs.io> (дата обращения: 24.05.2026). – Текст : электронный.